

Nivel 1: Ejercicios Sencillos (Conceptos Básicos)

Estos ejercicios se enfocan en una sola habilidad a la vez.

1. (Listas) Mi Primera Lista

- Crea una `List<String>` llamada `frutas` con tus 5 frutas favoritas.
- Imprime la segunda fruta de la lista (índice 1).
- Añade una nueva fruta al final de la lista.
- Elimina la primera fruta de la lista.
- Imprime la lista final.

2. (Mapas) Diccionario de Vehículo

- Crea un `Map<String, dynamic>` llamado `vehiculo` para representar un vehículo.
- Añade las siguientes claves y valores: `marca` (String), `modelo` (String), `anio` (int), `esElectrico` (bool).
- Imprime el `modelo` del vehiculo.
- Modifica el valor de `anio` a un año diferente.
- Añade una nueva clave `color` (String) con un valor.
- Imprime el mapa completo.

3. (Listas + `.where`) Filtrador de Números Pares

- Dada la lista `List<int> numeros = [1, 5, 8, 12, 15, 22, 30];`
- Usa el método `.where()` para crear una **nueva lista** llamada `pares` que contenga solo los números pares.
- Imprime la lista `pares`.

4. (Listas + `.map`) Duplicador de Números

- Usando la misma lista `numeros` del ejercicio anterior.
- Usa el método `.map()` para crear una **nueva lista** llamada `dobles` donde cada número de la lista original esté multiplicado por 2.
- Imprime la lista `dobles`.

5. (Funciones Flecha) Convertidor Rápido

- Toma el ejercicio 3 (`.where`) y el ejercicio 4 (`.map`) y reescríbelos usando la sintaxis de **función flecha** (`=>`) en lugar del bloque anónimo completo `(n) { return ...; }`.

6. (Clases y Atributos) Molde Básico de `Estudiante`

- Crea una clase llamada `Estudiante`.
- Añade 3 atributos (propiedades): `nombre` (String), `edad` (int) y `carrera` (String). Dale valores por defecto (ej: 'Sin nombre', 0, 'Sin carrera').

7. (Clases y Métodos) El Estudiante se Presenta

- A la clase `Estudiante` del ejercicio anterior, añade un método `void presentarse()` que imprima en consola: "Hola, soy [nombre], tengo [edad] años y estudio [carrera]."
- En tu `main()`, crea una instancia (objeto) de `Estudiante`, asigna valores a sus atributos y llama al método `presentarse()`.

Nivel 2: Ejercicios Intermedios (Combinando Conceptos)

Estos ejercicios requieren combinar dos o más conceptos de la clase.

8. (Constructores) Inicializando al `Estudiante`

- Modifica la clase `Estudiante` (ejercicio 6 y 7).
- Elimina los valores por defecto de los atributos.
- Crea un constructor que utilice el **atajo idiomático de Dart** (`this.atributo`) para inicializar `nombre`, `edad` y `carrera`.
- En `main()`, crea un estudiante pasando los valores directamente al constructor.

9. (Constructores Nombrados) El `Estudiante` Moderno

- Modifica el constructor de `Estudiante` (ejercicio 8) para que use **parámetros nombrados** (`{ }`).
- Haz que `nombre` y `carrera` sean `required`.
- Haz que `edad` sea opcional, con un valor por defecto de `18`.
- En `main()`, crea un estudiante usando la sintaxis de parámetros nombrados (ej: `nombre: 'Ana'`).

10. (Listas de Mapas) Lista de Tareas (JSON)

- Crea una `List<Map<String, dynamic>>` llamada `tareasJson`.
- Añade 3 mapas a la lista. Cada mapa representa una tarea y debe tener: `id` (int), `descripcion` (String) y `completada` (bool).
- Inventa los datos para las 3 tareas (asegúrate de que al menos una esté completada y otra no).

11. (Clases + `.fromMap`) Clase `Tarea`

- Crea una clase `Tarea` con los atributos: `id` (int), `descripcion` (String), `completada` (bool).
- Crea un constructor principal (con parámetros nombrados, como en el ejercicio 9).
- Crea un **constructor nombrado** llamado `Tarea.fromMap(Map<String, dynamic> map)` que reciba un mapa (como los del ejercicio 10) e inicialice los atributos de la clase.

12. (Listas + Clases + `.map`) De JSON a Objetos

- ¡El gran salto! Toma la lista `tareasJson` del ejercicio 10.
- Usa el método `.map()` para convertir esa `List<Map<String, dynamic>>` en una `List<Tarea>`.
- (Pista: Dentro del `.map()`, deberás llamar a tu constructor `Tarea.fromMap(...)`).
- Imprime la descripción de la primera tarea de tu nueva lista de objetos.

13. (Listas + Clases + `.where`) Filtrando Objetos

- Usando la `List<Tarea>` (la lista de objetos) del ejercicio 12.
- Usa `.where()` para crear una nueva lista llamada `tareasPendientes` que contenga solo los objetos `Tarea` cuyo atributo `completada` sea `false`.
- Imprime la cantidad de tareas pendientes.

14. (Inmutabilidad) Clase `Configuracion` Inmutable

- Crea una clase inmutable `Configuracion`.
- Debe tener 3 atributos: `final String urlApi`, `final String modo` (ej: 'dark', 'light'), `final int timeout`.
- Crea un constructor `const` que inicialice estos valores (usa parámetros nombrados y `required`).
- En `main()`, intenta crear una `Configuracion` y luego intenta modificar uno de sus atributos (debería darte un error).

Nivel 3: Ejercicios Complejos (Desafíos)

Estos ejercicios simulan problemas reales y requieren una integración profunda de todos los módulos.

15. (Inmutabilidad + `copyWith`) El Método `copyWith`

- A la clase inmutable `Configuracion` (ejercicio 14), añade el método `copyWith`.
- `Configuracion copyWith({String? urlApi, String? modo, int? timeout})`
- Este método debe devolver una **nueva instancia** de `Configuracion`, usando los valores nuevos si se proveen, o los valores actuales (`this.atributo`) si no se proveen.
- En `main()`, crea una `configInicial`. Luego, crea una `configModoOscuro` usando `configInicial.copyWith(modo: 'dark')`. Imprime ambas para ver que son diferentes.

16. (POO y Listas) Carrito de Compras

- Crea una clase `Producto` inmutable (`final String nombre, final double precio`).
- Crea una clase `CarritoDeCompras`.
- `CarritoDeCompras` debe tener un atributo: `List<Producto> _productos = []`.
- Añade un método `agregarProducto(Producto producto)`.
- Añade un método `double calcularTotal()` que itere sobre la lista `_productos` (puedes usar `.forEach` o un `for...in`) y devuelva la suma de todos los precios.
- Añade un `getter` `List<Producto> get productos => _productos;` para poder ver los productos desde fuera, pero no modificarlos directamente.

17. (POO Avanzado) Mejorando el Carrito

- (Investiga esto) Modifica `calcularTotal()` (ejercicio 16) para que use el método `.fold()` en la lista.
 - `_productos.fold(0.0, (total, producto) => total + producto.precio);`
- Modifica `agregarProducto` para que no reciba un `Producto`, sino (`String nombre, double precio`) y cree el objeto `Producto` *dentro* del método.

18. (Modularización) `part` y `part of`

- Crea un archivo `biblioteca_matematica.dart` que defina `library matematica;` y `part 'operaciones_privadas.dart';`.
- En `biblioteca_matematica.dart`, crea una función pública `int duplicarSuma(int a, int b)` que llame a una función privada `_sumar(int a, int b)`.
- Crea el archivo `operaciones_privadas.dart` que declare `part of matematica;`.

- Define la función `int _sumar(int a, int b) => a + b;` dentro de `operaciones_privadas.dart`.
- En tu `main.dart`, `import 'biblioteca_matematica.dart'` y prueba la función `duplicarSuma`.

19. (Desafío de Lógica) Procesador de Inventario

Dada una lista de mapas (JSON) de productos:

Dart

```
var jsonInventario = [
  {'nombre': 'Laptop', 'precio': 1500.0, 'stock': 10},
  {'nombre': 'Mouse', 'precio': 45.0, 'stock': 0},
  {'nombre': 'Teclado', 'precio': 120.0, 'stock': 50},
  {'nombre': 'Monitor', 'precio': 350.0, 'stock': 5},
];
```

-
- Crea una clase `Producto` con su constructor `.fromMap`.
- Escribe una función `List<String> obtenerNombresProductosCarosEnStock(List<Map<String, dynamic>> inventario)` que:
 1. Convierta la lista de mapas en una `List<Producto>`.
 2. Filtre esa lista para obtener solo productos con `stock > 0`.
 3. De los filtrados, filtre de nuevo para obtener solo productos con `precio > 100.0`.
 4. Finalmente, transforme esa lista de `Producto` en una `List<String>` que solo contenga los `nombres` de los productos.
- (Pista: ¡Puedes encadenar `.map().where().where().map()` !)

20. (Desafío Integrador) Mini Red Social

- Crea dos clases inmutables: `Usuario (final int id, final String nombre)` y `Comentario (final int id, final String texto, final int usuarioId)`.
- Crea una "base de datos" falsa: `List<Usuario> usuarios` y `List<Comentario> comentarios`.
- Escribe una función `List<String> obtenerComentariosFormateados(int postId)` (puedes ignorar el `postId` por ahora, solo usa la lista de comentarios).
- La función debe devolver una `List<String>` donde cada string tenga el formato: `"El usuario [Nombre del Usuario] dijo: [Texto del Comentario]"`.

- (Pista: Necesitarás `.map()` sobre la lista de comentarios. Dentro del `.map()`, necesitarás usar `.firstWhere()` sobre la lista de usuarios para encontrar el nombre del autor basado en el `usuarioId`).